

White Paper: Atomic Cryptographic Extension (ACE) — Hardware-Enforced Stateful Cryptography

Author: Roberto Avanzi

Version v0.2.0, 2026-06-18: Draft

Executive Summary

Modern high-throughput cloud and client workloads demand low-latency, high-performance cryptography. However, existing hardware acceleration mechanisms force a compromise: general-purpose CPU instructions (e.g., AES-NI) offer high performance but expose raw keys and intermediate states in memory, while secure enclaves (TEEs) and Hardware Security Modules (HSMs) protect secrets but introduce severe performance penalties due to context-switch overhead, cache pollution, and non-uniform interfaces.

To resolve this fundamental tension, we propose the **Atomic Cryptographic Extension (ACE)**, an ISA-level architectural framework designed around the concept of **Cryptographic Contexts (CCs)**. A Cryptographic Context is an opaque, hardware-isolated data structure that encapsulates cryptographic keys, algorithm state, and usage policies. ACE provides hardware-enforced protection of these contexts within dedicated **Cryptographic Registers (CRs)**, and secure export to memory via **Sealed Cryptographic Contexts (SCCs)**, and later import of SCCs into CRs, using authenticated encryption.

By executing cryptographic operations atomically at the primitive level (e.g., full block encryptions or hash rounds) and referencing opaque Cryptographic Registers instead of raw keys, ACE significantly reduces key exposure in memory and prevents state-manipulation attacks. ACE provides a uniform, algorithm-agnostic ISA interface that scales across heterogeneous cores, preventing instruction set bloat while delivering HSM-grade security at CPU-instruction speeds.

We call on the industry to standardize Cryptographic Context formats and semantics to enable secure, seamless cryptographic interoperability across heterogeneous architectures.

The Limits of Existing Approaches

— Specialized Environments (HSMs and TEEs)

Historically, high cryptographic throughput has been achieved through the use of accelerators, for instance the IBM 4758/4764/4765 Cryptographic Modules ([Smith & Weingart, 1999](#); [Dyer et al., 2001](#)) or the Sun Cryptographic Accelerator PCI boards ([Hung & Bhattacharya, 2002](#)). Versions with high security of such accelerators exist as well, such as hardened modules ([Attridge, 2021](#)) and secure processors for embedded applications, which can provide strong protection for cryptographic keys,

While highly secure, and often fast, these systems suffer from significant invocation latency and setup overhead. To reduce this latency, modern SoCs can rely on Trusted Execution Environments (TEEs) or secure enclaves, e.g., ARM TrustZone ([Alves & Felton, 2004](#)), Intel SGX ([Intel, 2015](#)), and perform cryptographic operations directly in the secure environment.

However, TEEs still introduce substantial performance penalties from context switches, memory encryption/decryption overhead ([Gueron, 2016](#)), cache pressure, and increased memory access latency. In cloud-scale systems executing millions of operations per second, these overheads degrade throughput and increase operational costs.

— General-Purpose Environments & Key-Hiding ISAs

To bypass the overheads of separate cryptographic accelerators, general-purpose ISAs introduce cryptographic instructions (e.g., Intel AES-NI ([Gueron, 2012](#))). While fast, these instructions process keys in standard registers, exposing them to memory disclosure vulnerabilities ([van der Veen et al., 2012](#); [Lou et al., 2022](#)) and cold boot attacks ([Halderman et al., 2009](#)).

Hardware-assisted key-hiding mechanisms like Intel KeyLocker ([Intel, 2020](#)) and IBM Protected Key ([IBM, 2025](#)) attempt to mitigate this by operating on encrypted keys decrypted internally by hardware. However, **hiding the key is insufficient to guarantee cryptographic security**.

Complex modes of operation, such as XTS ([Dworkin, 2010](#)), GCM-SIV ([Gueron et al., 2019](#)), and OCB ([Krovetz & Rogaway, 2021](#)), to make a few examples, rely on the secrecy of derived keys, initialization vectors (IVs), and intermediate states (such as authentication tags). Exposing these intermediate values allows attackers to bypass confidentiality or forge messages without recovering the master key. Key-hiding-only ISAs fail to protect this runtime algorithmic state.

— Instruction Set Proliferation

Furthermore, the traditional practice of adding dedicated instructions for every new cryptographic primitive or standard (e.g., national algorithms, post-quantum cryptography) is unsustainable. This leads to instruction set bloat, which would be particularly problematic in RISC architectures with tight encoding spaces. A scalable architecture must decouple the ISA from specific cryptographic primitives, providing a constant instruction footprint regardless of the underlying algorithms supported.

A Comprehensive Approach

Instead of treating cryptography as stateless operations on raw keys, ACE models cryptographic execution as stateful operations on **Cryptographic Contexts (CCs)**.

— Cryptographic Contexts (CCs)

A CC is an opaque, hardware-isolated object that encapsulates:

1. **Key Material:** Master and derived keys;
2. **Metadata:** Usage policies, algorithm identifiers, and environmental constraints (e.g., hardware configuration, device, privilege level, security state, as well as physical or virtual boot cycle); and
3. **Algorithm State:** Intermediate values, IVs, and state machine stage.

Once provisioned, a CC's internal structure is completely opaque to software. It resides in dedicated, non-addressable **Cryptographic Registers** and can only be manipulated via well-defined architectural instructions. Only the metadata and certain algorithm state information can be made visible to software, while the rest, including key material and other internal derived state are kept hidden within the hardware, or exported in a secure way—by means of authenticated encryption.

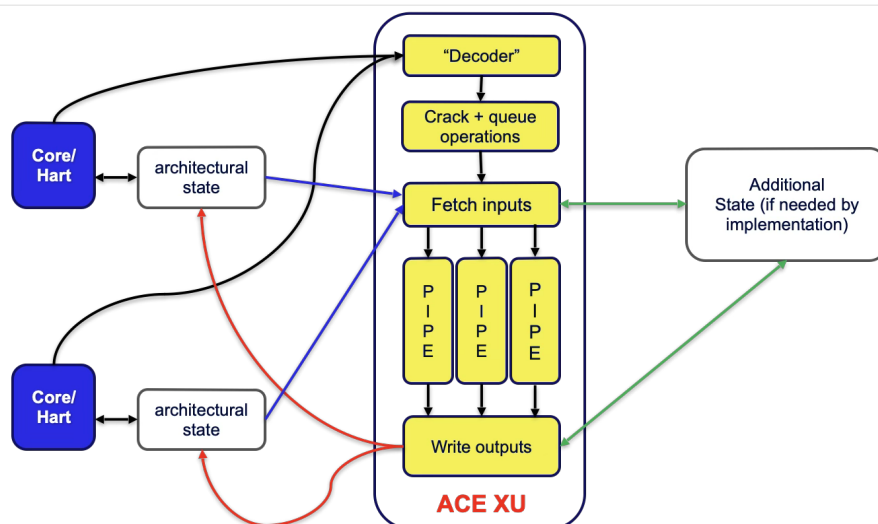
— ACE Instructions and the ACE Unit

ACE decouples the ISA from specific algorithms by defining a minimal, constant set of instructions. The two primary instructions are:

- **execute:** Performs a unit of algorithmic work (e.g., block encryption, hash absorption).
- **change state:** Transitions the cryptographic state machine between phases (e.g., from AAD processing to ciphertext processing in AEAD).

These instructions are executed **atomically** at the primitive level (e.g., full AES or Keccak rounds). Primitives are never split into individual rounds in software, eliminating side-channel leakage of intermediate round states ([Bouillaguet et al., 2012](#); [Bar-On et al., 2018](#)).

The ACE hardware unit can be implemented as a closely coupled core pipeline or as an attached accelerator shared among multiple heterogeneous cores, maintaining independent architectural state per hardware thread. The figure below illustrates an attached implementation: A separate architectural state is maintained for each hardware thread, while the unit itself is shared and can interleave pipelined operations from different requesters.



— Lifecycle and Context Switching

To support context switching and virtualization, a CC can be exported to memory as a **Sealed Cryptographic Context (SCC)**. The hardware encrypts and authenticates the CC using a hardware-managed **Sealing Key** before writing it to memory, preserving all metadata and policy constraints. The Sealing Key can be reconfigured by Machine-mode software, enabling secure cryptographic isolation between process domains and seamless virtual machine (VM) migration.

Advantages

By design, ACE eliminates traditional cryptographic attack surfaces while maximizing performance:

- **Zero-Exposure Security:** Keys, derived keys, and intermediate states never appear in software-visible registers or memory, neutralizing memory disclosure, cold boot, and transient execution attacks.
- **State-Machine Integrity:** Hardware-enforced state transitions prevent misuse or out-of-order execution of cryptographic phases (e.g., reusing IVs or bypassing authentication checks).
- **Side-Channel Mitigation:** Because algorithm execution is encapsulated within the ACE unit, hardware-level masking and countermeasures can be implemented transparently. Software requires no modification to benefit from these protections.
- **Elimination of Context-Switch Overhead:** High-performance cryptographic operations run in the application's context without requiring costly switches to TEEs or secure enclaves.
- **Algorithm Agility & Future-Proofing:** The ISA remains constant. New algorithms (including Post-Quantum Cryptography) can be supported by updating the ACE unit (either the hardware, or its microcode) without adding new CPU instructions.

Call for Action

In modern heterogeneous cloud environments, cryptographic workloads must span multiple instruction set architectures (ISAs) and hardware platforms. To prevent fragmentation and enable secure interoperability, the industry must converge on common standards:

1. **Unified Formats:** Standardize the binary representations for Cryptographic Register initialization and Sealed Cryptographic Contexts (SCCs) to allow secure migration across heterogeneous systems.
2. **Semantic Specifications:** Establish a shared specification for cryptographic state machines, algorithm behaviors, and metadata policies to ensure consistent enforcement across different hardware implementations.

Conclusion

The Atomic Cryptographic Extension (ACE) redefines hardware-accelerated cryptography by shifting from stateless key-hiding to stateful, policy-enforced execution. By encapsulating keys, state, and metadata into hardware-isolated Cryptographic Contexts, ACE delivers HSM-grade security at the speed of native CPU instructions. Standardizing these interfaces across the industry will pave the way for secure, high-performance, and agile cryptographic infrastructure across heterogeneous cloud and client systems.

Bibliography

Alves, T., & Felton, D. (2004). Trustzone: Integrated hardware and software security – ARM white paper. *Information Quarterly*, 3, 18–24.

Attridge, J. (2021). *An Overview of Hardware Security Modules*. The Escal Institute of Advanced Technologies, Inc. d/b/a SANS Institute. <https://www.sans.org/white-papers/757>

Bar-On, A., Dunkelman, O., Keller, N., Ronen, E., & Shamir, A. (2018). Improved Key Recovery Attacks on Reduced-Round AES with Practical Data and Memory Complexities. In H. Shacham & A. Boldyreva (Eds.), *Advances in Cryptology - CRYPTO 2018 - 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2018, Proceedings, Part II* (pp. 185–212). Springer. https://doi.org/10.1007/978-3-319-96881-0_7

Bouillaguet, C., Derbez, P., Dunkelman, O., Fouque, P.-A., Keller, N., & Rijmen, V. (2012). Low-Data Complexity Attacks on AES. *IEEE Trans. Inf. Theory*, 58(11), 7002–7017. <https://doi.org/10.1109/TIT.2012.2207880>

Dworkin, M. (2010). Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices. In *Federal information processing standards (FIPS), National Institute of Standards and Technology, Gaithersburg, MD*. <https://csrc.nist.gov/pubs/sp/800/38/e/final>

Dyer, J. G., Lindemann, M., Perez, R., Sailer, R., van Doorn, L., Smith, S. W., & Weingart, S. (2001). Building the IBM 4758 Secure Coprocessor. *Computer*, 34(10), 57–66. <https://doi.org/10.1109/2.955100>

Gueron, S. (2012). *Intel® Advanced Encryption Standard (Intel® AES) Instructions Set - Rev 3.01*. <https://www.intel.com/content/www/us/en/developer/articles/tool/intel-advanced-encryption-standard-aes-instructions-set.html>

Gueron, S. (2016). A Memory Encryption Engine Suitable for General Purpose Processors. *IACR Cryptology EPrint Archive*. <http://eprint.iacr.org/2016/204>

Gueron, S., Langley, A., & Lindell, Y. (2019). *AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption*. <http://www.ietf.org/rfc/rfc8452.txt>

Halderman, J. A., Schoen, S. D., Heninger, N., Clarkson, W., Paul, W., Calandrino, J. A., Feldman, A. J., Appelbaum, J., & Felten, E. W. (2009). Lest we remember: cold-boot attacks on encryption keys. *Commun. ACM*, 52(5), 91–98. <https://doi.org/10.1145/1506409.1506429>

Hung, S.-H., & Bhattacharya, P. (2002). *On the Delivered Performance of the Sun Crypto Accelerator 1000 (Sun User Performance Group Conference, Honolulu)*. <https://www.csie.ntu.edu.tw/~hungsh/2002%20SUPERG.pdf>

IBM. (2025). *z/Architecture Principles of Operation, 15th Edition*. https://www.ibm.com/docs/en/module_1678991624569/pdf/SA22-7832-14.pdf

Intel. (2015). *Intel® Software Guard Extensions*. <https://software.intel.com/en-us/isa-extensions/intel-sgx>

-
- Intel. (2020). *Intel® Key Locker Specification*. <https://www.intel.com/content/www/us/en/content-details/671438/intel-key-locker-specification.html>
- Krovetz, T., & Rogaway, P. (2021). The Design and Evolution of OCB. *J. Cryptol.*, 34(4), 36. <https://doi.org/10.1007/S00145-021-09399-8>
- Lou, X., Zhang, T., Jiang, J., & Zhang, Y. (2022). A Survey of Microarchitectural Side-channel Vulnerabilities, Attacks, and Defenses in Cryptography. *ACM Comput. Surv.*, 54(6), 122:1–122:37. <https://doi.org/10.1145/3456629>
- Smith, S. W., & Weingart, S. (1999). Building a high-performance, programmable secure coprocessor. *Computer Networks*, 31(8), 831–860. [https://doi.org/https://doi.org/10.1016/S1389-1286\(98\)00019-X](https://doi.org/https://doi.org/10.1016/S1389-1286(98)00019-X)
- van der Veen, V., dutt-Sharma, N., Cavallaro, L., & Bos, H. (2012). Memory Errors: The Past, the Present, and the Future. In D. Balzarotti, S. J. Stolfo, & M. Cova (Eds.), *Research in Attacks, Intrusions, and Defenses - 15th International Symposium, RAID 2012, Amsterdam, The Netherlands, September 12-14, 2012. Proceedings* (pp. 86–106). Springer. https://doi.org/10.1007/978-3-642-33338-5_5